

The Direction Chase

Three lamps race, reverse, and shut themselves down

Motor logic you can see, with no motor or driver connected.

Lesson	020 — component	Build time	about 20 minutes
Prerequisites	Lessons 001–019	Evidence	three resistor-limited lamps
Board	Arduino Mega 2560	Power	USB only
Visible result	two direction rounds	Finish	automatic at 10 seconds

1 What you are making

Three LEDs stand in for the inputs of a future motor driver. Forward and reverse are direction lamps. Enable is the “go” lamp: its brightness changes with PWM duty. On every reversal all three go dark for 250 ms before the other direction may light.

The whole lesson is inert. Use only the three LEDs and resistors shown here. Do not connect an H-bridge, motor, motor supply, relay, or wheel. The Elegoo family listing does not establish an exact driver or motor electrical identity, so powered motion is outside this lesson.

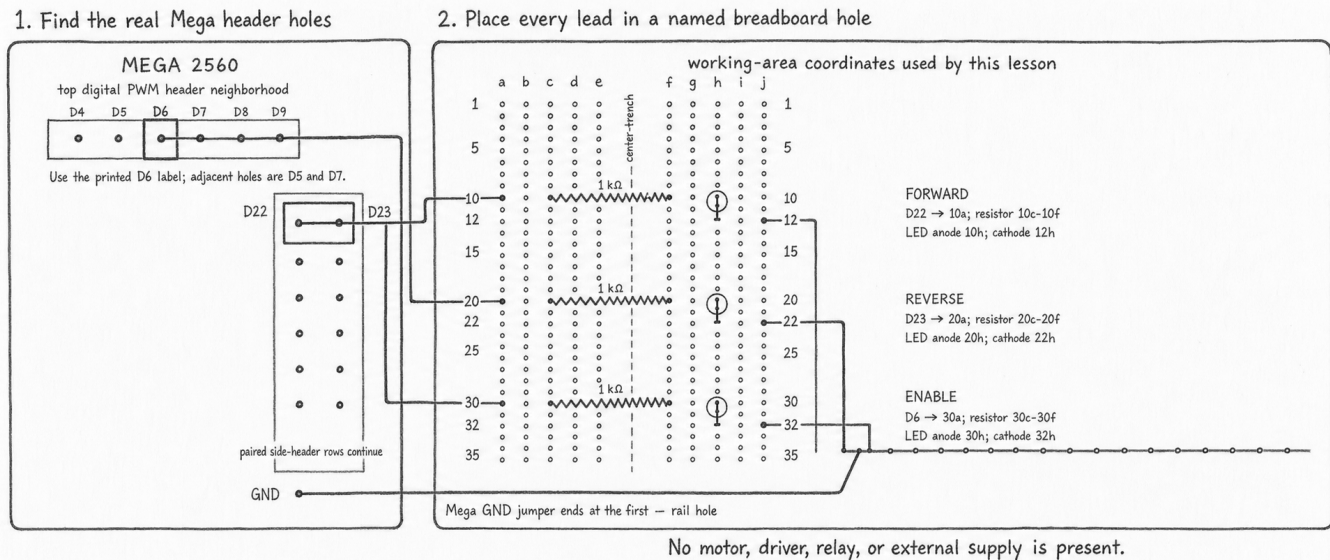
2 Parts

Part	Use
Arduino Mega 2560	runs the fixed ten-second show
Three ordinary LEDs	forward, reverse, and enable lamps
Three 1 kΩ resistors	one series resistor for each LED
Breadboard and jumpers	the three independent signal paths

At a conservative 5 V rail and 2 V LED drop, each continuously lit path is about $(5 - 2)/1000 = 3$ mA. Unplug USB before changing a wire. If an LED is hot, the USB connection is unstable, or the picture and your board disagree, unplug and inspect the circuit.

3 Find the exact pins

On the Mega's long paired digital header, find D22 and D23 next to each other. D6 is on the PWM-numbered digital header near the board end. Use any labeled GND pin. The drawing separates these locations so the labels remain readable.



A pencil-style layout of the Mega paired header and breadboard. D22, D23, and D6 each run through their own 1 k Ω resistor to an LED long leg. All three short legs return through the breadboard negative rail to Mega GND. No motor or driver appears in the circuit.

Lamp	Mega pin	Complete electrical path
Forward / TP-F	D22	D22 → 1 k Ω → LED long leg; short leg → negative rail
Reverse / TP-R	D23	D23 → 1 k Ω → LED long leg; short leg → negative rail
Enable / TP-E	D6	D6 → 1 k Ω → LED long leg; short leg → negative rail
Return	GND	Mega GND → negative rail

4 Build in three payoffs

First payoff — direction. Build only D22 and its LED path, then add D23 and its path. The two LEDs must have separate resistors. A shared resistor would couple the signals and spoil the chase.

Second payoff — speed. Add D6 and the enable LED. This lamp starts dim, becomes bright, and later runs bright in both directions. That visible change comes from two same-direction duty commands, not from a delay.

Third payoff — reversal. Upload the canonical sketch. Watch closely when forward hands the chase to reverse: every lamp goes dark before reverse and enable return. The reverse round cannot appear correctly unless both direction paths and enable are independent.

5 Read the ten-second show

	Time	Direction lamps	Enable lamp	What the controller is doing
	0.00–0.75 s	both dark	dark	stopped and ready
	0.75–1.75 s	forward	dim	forward at duty 72
	1.75–2.75 s	forward	bright	same direction, duty 180
	2.75 s	both dark	dark	250 ms reversal dead time
about	3.00–4.50 s	reverse	bright	reverse at duty 160
	4.50–5.50 s	both dark	dark	stopped between rounds
	5.50–6.50 s	forward	bright	second round
	6.50 s	both dark	dark	another 250 ms dead time
about	6.75–8.00 s	reverse	bright	second reversal completed
	8.00–10.00 s	both dark	dark	stopped
	10.00 s onward	all dark	dark	automatic safe shutdown

The blackout is the important trick. The requested direction may already be reverse while the *applied* command remains stopped. The opposite output is allowed only after the explicit deadline.

```
void loop ()
{
    const adk::TimePoint now (millis ());

    observeOperatorRequest (now);
    decideMotorIntent      (now);
    actuateMotorEvidence   ();
}
```

The loop always observes, decides, then actuates the applied snapshot. There is no blocking delay. Stop cancels pending motion, no state permits both direction signals high, and shutdown first requests stopped intent before releasing the pins.

6 Make the idea your own

1. Swap the physical forward and reverse LEDs. Predict how the show will look while the code's meaning stays unchanged.
2. Change the first duty from 72 to 36. The enable lamp should begin even dimmer, while direction remains forward.
3. Change the dead time from 250 ms to 500 ms. The deliberate blackout becomes much easier to count.
4. Add a third round that requests stop during the blackout. A correct controller stays dark instead of restarting at the old deadline.

7 If the chase looks wrong

What you see	Inspect while USB is unplugged
One direction never lights	its LED polarity and its pin–resistor–LED path
Both directions light together	accidental shared rows or a D22/D23 short
Direction works but enable is dark	D6 path, LED polarity, and resistor row
No brightness change	confirm the enable jumper really starts at PWM pin D6
No dark reversal gap	confirm you uploaded the Lesson 020 sketch
Lights remain after 10 seconds	reset once; if repeated, disconnect USB

8 What the repository checks behind the scenes

Automated tests replay stopped, same-direction speed changes, both reversals, stop during dead time, exact deadline edges, time wraparound, invalid commands, rollback, repeated shutdown, and resource reuse. The Mega build also checks the sketch’s flash and RAM budget. Those checks protect the lesson; they are not extra chores for the learner.

9 Where this leads

A real motor needs an exactly identified motor, driver, primary datasheets, measured stall current, separate current-limited load supply, complete flyback path, physical power disconnect, and guarded unloaded fixture. None of those facts follows from a retail kit-family name. Until that separate E2 work is qualified, these three lamps are the complete supported circuit.

Status: host verified; physical observation remains unclaimed. Current reference material and the canonical sketch are published with the HTML lesson.